SESI PRAKTIKUM

Membangun Class Pertamamu di Python

Praktik langsung: Class, Object, Attribute, dan Method dasar

Sub-CPMK 1 · Mata Kuliah Pemrograman Berorientasi Objek

TUJUAN

Apa yang Akan Anda Bisa Lakukan?

- ✓ Membuat sebuah Class Python dari nol, lengkap dengan constructor __init__
- ✓ Mendefinisikan Attribute untuk menyimpan data pada sebuah object
- ✓ Mendefinisikan Method untuk memberi perilaku pada sebuah object
- ✓ Membuat beberapa Object dari satu Class yang sama dan menggunakannya

PERSIAPAN

Yang Perlu Disiapkan Sebelum Mulai

1

Python 3.x

Pastikan Python sudah terpasang. Cek dengan perintah 'python --version' di terminal.

2

Editor / IDE

Gunakan VS Code, PyCharm, atau editor teks favorit untuk menulis kode .py

3

Terminal

Digunakan untuk menjalankan file Python dengan perintah 'python nama_file.py'

Verifikasi cepat: buka terminal, ketik `python3 --version` lalu tekan Enter.

ALUR KERJA

6 Langkah Membangun Class Pertamamu

1. Buat Class Kosong
2. Constructor __init__
3. Tambah Attribute
4. Tambah Method
5. Buat Object
6. Akses & Panggil

LANGKAH 1

Membuat File & Class Kosong

Buat file baru bernama mahasiswa.py, lalu definisikan sebuah class kosong menggunakan kata kunci class.

- Nama class sebaiknya diawali huruf kapital (PascalCase)
- pass digunakan sebagai placeholder agar class tidak kosong secara sintaks
- Simpan file dengan ekstensi .py

```
# file: mahasiswa.py

class Mahasiswa:
    pass


# class ini belum punya attribute
# atau method apa pun (masih kosong)
```

LANGKAH 2

Constructor __init__ dan Parameter self

`__init__` adalah method khusus yang otomatis dijalankan setiap kali sebuah Object baru dibuat. Parameter `self` selalu ada di posisi pertama, dan merujuk ke object yang sedang dibuat.

```
class Mahasiswa:
    def __init__(self, nama, nim):
        pass

# self  -> merujuk object itu sendiri
# nama, nim -> data yang dikirim saat
#              object dibuat
```

Catatan Penting

- self tidak perlu dikirim manual — Python mengisinya otomatis
- Anda boleh memberi nama lain, tapi 'self' adalah konvensi standar
- `__init__` ditulis dengan double underscore di kedua sisi

LANGKAH 3

Menambahkan Attribute

Simpan parameter yang diterima constructor ke dalam **`self.nama_attribute`** agar datanya melekat pada object.

```
class Mahasiswa:
    def __init__(self, nama, nim, ipk):
        self.nama = nama
        self.nim = nim
        self.ipk = ipk

# nama, nim, ipk kini menjadi ATTRIBUTE
# yang melekat pada setiap object
```

Alur Data

1. Parameter nama diterima constructor
2. Disimpan ke self.nama
3. Melekat selama object hidup

LANGKAH 4

Menambahkan Method

Tambahkan fungsi di dalam class (dengan parameter **`self`**) untuk memberi object sebuah perilaku, misalnya menampilkan datanya sendiri.

```
class Mahasiswa:
    def __init__(self, nama, nim, ipk):
        self.nama = nama
        self.nim = nim
        self.ipk = ipk

    def tampilkan_data(self):
        print(f"{self.nama} - {self.nim}")
        print(f"IPK: {self.ipk}")
```

Ingat

- Method didefinisikan dengan **`def`** di dalam class, sejajar dengan __init__
- self wajib ada sebagai parameter pertama, agar method bisa membaca self.nama, self.nim, dst
- Method boleh punya parameter tambahan selain self

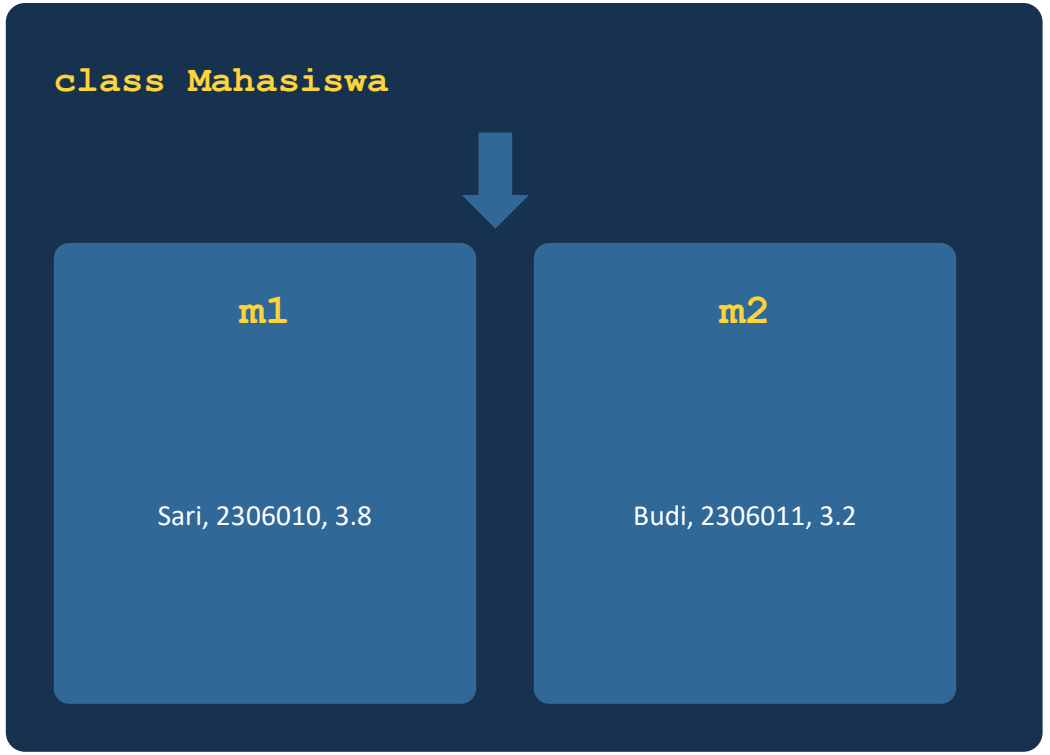
LANGKAH 5

Membuat Object (Instansiasi)

Panggil nama Class seperti fungsi, dengan argumen sesuai parameter __init__ (selain self), untuk membuat Object baru.

```
# membuat dua object berbeda
m1 = Mahasiswa("Sari", "2306010", 3.8)
m2 = Mahasiswa("Budi", "2306011", 3.2)

print(type(m1))
# <class '__main__.Mahasiswa'>
```



LANGKAH 6

Mengakses Attribute & Memanggil Method

Gunakan tanda titik (.) setelah nama object untuk mengakses attribute atau memanggil method-nya.

```
# mengakses attribute
print(m1.nama)
print(m1.ipk)

# memanggil method
m1.tampilkan_data()
m2.tampilkan_data()

# mengubah attribute langsung
m1.ipk = 3.9
```

`object.attribute` **vs** **`object.method()`**

`m1.nama` → mengambil nilai data (tanpa tanda kurung)

`m1.tampilkan_data()` → menjalankan aksi (pakai tanda kurung)

Kesalahan umum: lupa tanda kurung () saat memanggil method.

STUDI KASUS

Program Lengkap: Class Mahasiswa

```
class Mahasiswa:
    def __init__(self, nama, nim, ipk):
        self.nama = nama
        self.nim = nim
        self.ipk = ipk

    def tampilkan_data(self):
        print(f"{self.nama} ({self.nim})")
        print(f"IPK: {self.ipk}")

    def is_lulus(self):
        return self.ipk >= 2.0

m1 = Mahasiswa("Sari", "2306010", 3.8)
m1.tampilkan_data()
print("Lulus?", m1.is_lulus())
```

OUTPUT TERMINAL

```
Sari (2306010)
IPK: 3.8
Lulus? True
```

Perhatikan: is_lulus() mengembalikan nilai boolean berdasarkan attribute ipk milik object itu sendiri.

LATIHAN MANDIRI

Sekarang Giliran Anda: Class Buku

Buatlah class Buku dengan attribute judul, penulis, dan jumlah_halaman, serta method tampilkan_info() yang mencetak ketiga attribute tersebut.

1. Definisikan class Buku dan constructor __init__ dengan 3 parameter
2. Simpan ketiga parameter sebagai attribute menggunakan self
3. Tambahkan method tampilkan_info() yang mencetak semua attribute
4. Buat 2 object Buku dengan judul berbeda, lalu panggil tampilkan_info() masing-masing
5. Tantangan tambahan: buat method is_tebal() yang True jika jumlah_halaman > 300

DEBUGGING

Kesalahan Umum & Cara Memperbaikinya

! Lupa self

```
TypeError: __init__() missing 1 required positional argument
```

Solusi: Tambahkan self sebagai parameter pertama di setiap method

! Salah indentasi

```
IndentationError: expected an indented block
```

Solusi: Pastikan isi method/class menjorok konsisten (4 spasi)

! Lupa tanda kurung ()

```
Method tidak berjalan, hanya menampilkan referensi fungsi
```

Solusi: Tulis object.method() bukan object.method

! Attribute belum dibuat

```
AttributeError: 'Mahasiswa' object has no attribute 'ipk'
```

Solusi: Pastikan self.ipk diisi di dalam __init__